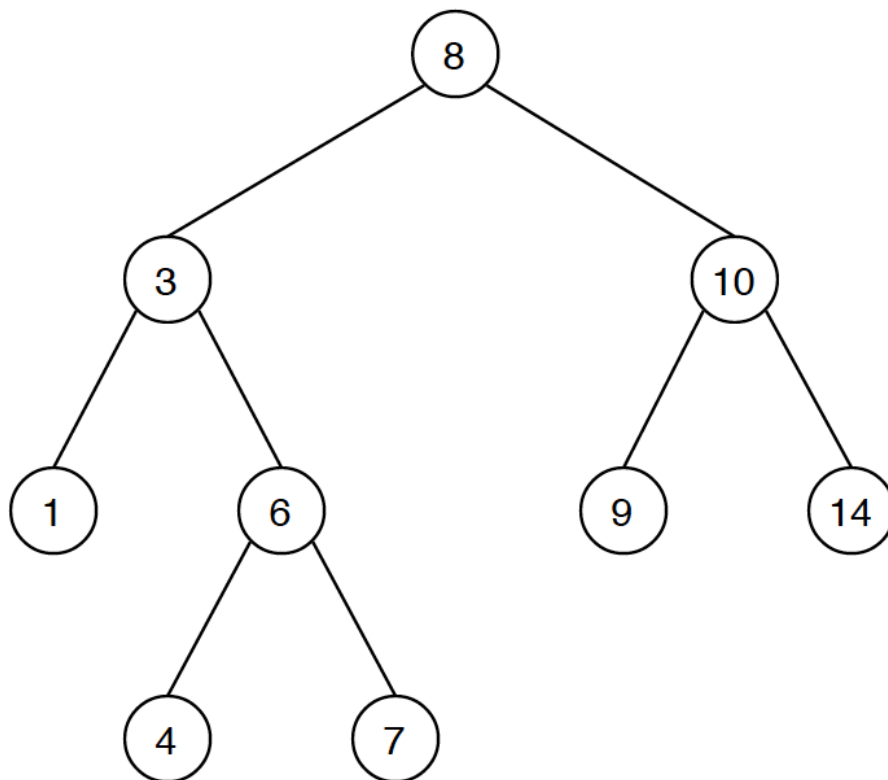


Бинарные деревья поиска

Во-первых, это **бинарное** дерево:

Такое дерево может быть представлено при помощи связанной структуры данных, в которой каждый узел является объектом.

```
type Node struct {  
    Parent *Node  
    Left  *Node  
    Right *Node  
    Key   int  
}
```



Ключи в бинарном дереве поиска хранятся таким образом, чтобы в любой момент удовлетворять **СВОЙСТВУ БИНАРНОГО ДЕРЕВА ПОИСКА**:

Если x – узел бинарного дерева поиска, а узел y находится в левом поддереве x , то

$$\text{key}[y] \leq \text{key}[x]$$

Если узел y находится в правом поддереве x , то

$$\text{key}[x] \leq \text{key}[y]$$

Обход дерева (Inorder Tree Walk)

Свойство бинарного дерева позволяет вывести все ключи в отсортированном порядке с помощью простого рекурсивного алгоритма – **ЦЕНТРИРОВАННОГО (СИММЕТРИЧНОГО) ОБХОДА ДЕРЕВА** (Inorder tree walk):

```
func InorderTreeWalk(x *Node) {  
    if x != nil {  
        InorderTreeWalk(x.Left)  
        fmt.Println(x.Key)  
        InorderTreeWalk(x.Right)  
    }  
}
```

Асимптотическая сложность алгоритма: $\Theta(n)$

Потому что после начального вызова процедура вызывается ровно два раза для каждого узла дерева – по разу для левого и правого узлов.

ТЕОРЕМА:

Если x – корень поддерева, в котором имеется n узлов, то процедура $InorderTreeWalk(x)$ выполняется за время $\Theta(n)$.

ДОКАЗАТЕЛЬСТВО Если же потом добавлю из Кормена, мне контекст проходить надо баля...

Поиск

```
func TreeSearch(x *Node, k int) *Node {
    if x == nil || k == x.Key {
        return x
    }

    if k < x.Key {
        return TreeSearch(x.Left, k)
    } else {
        return TreeSearch(x.Right, k)
    }
}
```

А теперь запишем итеративный поиск:

```
func IterativeTreeSearch(x *Node, k int) *Node {
    var temp *Node
    temp = x
    for temp != nil && k != temp.Key {
        if k < temp.Key {
            temp = temp.Left
        } else {
            temp = temp.Right
        }
    }
    return temp
}
```

Поиск минимума и максимума

Ну, будем честны, зная свойство найти минимум и максимум предельно легко)

```
func TreeMin(x *Node) *Node {
    temp := x
    for temp.Left != nil {
        temp = temp.Left
    }
    return temp
}
```

Аналогично для поиска максимального:

```
func TreeMax(x *Node) *Node {
    temp := x
    for temp.Right != nil {
        temp = temp.Right
    }
    return temp
}
```

Предшествующий и последующий элементы

Иногда, имея узел в бинарном дереве поиска, требуется определить, какой узел следует за ним в отсортированной последовательности.

Скажем у нас дерево это станции метро, причем их ключи – это ничто иное как важность географического расположения или личностной оценки (условно говоря там куча тцшек, парков и тп)

И вот сначала надо найти ближайший по пользе к текущему, а затем построить кратчайший маршрут... (лан, пример хуйня)

```
func TreeSuccessor(x *Node) *Node {
    temp := x
    if temp.Right != nil {
        return TreeMin(temp.Right)
    }
    temp2 := x.Parent
    for temp2 != nil && temp == temp2.Right {
        temp = temp2
        temp2 = temp2.Parent
    }
    return temp2
}
```

ОБЪЯСНЕНИЕ:

Крч, если `x.Right` не пустой, то просто возвращаем минимальный элемент из `x.Right`.

А если `x.Right` пустой, то очевидно, что `x.Left` нам по определению не подходит и мы переходим к `x.Parent`. Если правый от `x.Parent` это наш `x`, то повторяем нашу операцию, тк:

искомый элемент не может быть в левой части от `x` или его родных, тк правые дети по определению больше левых детей, значит, наш искомый элемент лежит в более правой ветке и если

`x.Parent.Right != x`, то `x.Parent` искомый элемент.

ТЕОРЕМА:

Операции поиска, определения минимального и максимального элемента, а также предшествующего и последующего, в бинарном дереве поиска высоты h могут быть выполнены за время $O(h)$.